Bash Cheat Sheet

10-Bash_Programming.pdf
PDF

make a detailed cheat sheet for this bash lecture file

Here's a **detailed Bash Cheat Sheet** based on the lecture from the PDF you shared. It's structured by topics and includes syntax, tips, and example snippets.

🧠 Bash Programming Cheat Sheet

🔹 Getting Started

Bash Script Syntax

```bash
#!/usr/bin/env bash  # Preferred shebang
```

- `chmod +x script.sh` → Make script executable
- Run with `./script.sh` or `bash script.sh`

🔹 Printing Output

```bash
echo "Hello"
echo -e "Line1\nLine2\tTabbed"
echo -e "\e[32mGreen Text\e[0m"
```

- `>> file.txt` → Appends to a file

- `\e[31m` = red, `\e[32m` = green, `\e[34m` = blue

🔹 Variables

```bash
name="Alice"
echo "Hello, $name"
num1=5
num2=10
echo "$((num1 + num2))"
```

- No spaces around `=`
- `${var}` helps avoid ambiguity: `echo "${name}123"`

🔹 Quotes

- `" "` → Expands variables
- `' '` → Literal (no expansion)
- `\` → Escape special characters

🔹 Command Substitution

```bash
now=$(date)
files=`ls`
```

🔹 Environment Variables

```bash
echo $HOME $USER $PATH
env  # List all environment variables
```

🔹 Arithmetic

```bash
let "a = 5 + 3"
a=$((5 + 3))
echo "scale=2; 5/2" | bc -l  # Floating point
declare -i x=5  # Integer only
```

🔹 Operators

Arithmetic: `+ - * / % **`

Comparison:

```bash
-eq  -ne  -gt  -lt  -ge  -le
```

Logical:

```bash
&&  ||  !
```

String:

```bash
=  !=  -z (empty)  -n (not empty)
```

File Tests:

```bash
-e file      # exists
-f file      # regular file
-d dir       # directory
-r/-w/-x     # read/write/execute
```

Assignment:

```bash
((x+=1))  ((x-=1))  ((x*=2))
```

Bitwise:

```bash
& | ^ ~ << >>
```

🔹 Conditionals

```bash
if [[ $x -gt 5 ]]; then
  echo "Greater than 5"
elif [[ $x -eq 5 ]]; then
  echo "Equal to 5"
else
  echo "Less than 5"
fi
```

🔹 Loops

`for` Loop

```bash
for i in {1..5}; do echo $i; done
for file in *.sh; do echo "$file"; done
```

`while` Loop

```bash
count=1
while (( count <= 5 )); do
  echo $count
  ((count++))
done
```

`until` Loop

```bash
until (( count > 5 )); do ...; done
```

🔹 Special Variables

Variable	Description
`$0`	Script name
`$1`	First argument
`$#`	Number of arguments
`$@`	All arguments (separate)
`$*`	All arguments (single string)
`$$`	Script PID

Variable	Description
`$?`	Exit status of last command
`$_`	Last argument of previous command

🔹 Functions

bash

```
greet() {
  echo "Hello, $1"
}

greet "Alice"

# Return values
result=$(factorial 5)

# Function with return
check_file() {
  [[ -f "$1" ]] && return 0 || return 1
}
```

🔹 Local vs Global Variables

bash

```
message="global"
modify() {
  local message="local"
  echo $message
}
```

- Use `export VAR=value` to pass to child processes

🔹 File Reading

Method 1

```bash
res=$(cat file.txt)
```

Method 2

```bash
while read -r line; do echo "$line"; done < file.txt
```

Method 3 (With Argument)

```bash
if [[ $# -ne 1 ]]; then echo "Usage: $0 <file>"; exit 1; fi
while read -r line; do echo "$line"; done < "$1"
```

🔹 File Writing

```bash
cat <<EOF > file.txt
First line
Second line
EOF

echo "Line" >> file.txt
printf "Name: %s, Score: %.2f\n" "John" 92.35 >> file.txt
```

🔹 Arrays

Indexed Arrays

```bash
arr=("one" "two")
arr[2]="three"
echo ${arr[0]}
echo ${#arr[@]}  # length
```

Associative Arrays (Bash 4+)

```
```